Kaggle Dataset Analysis Workspace

Architecture reference — how the app actually works, and where it's held together with tape. Written for your own future reference, not for sharing.

---

1. What this is, in one paragraph

A FastAPI backend plus a single-file vanilla-JS frontend that lets someone search Kaggle datasets, profile them without ever fully downloading them, ask natural-language questions that get answered with real pandas/scipy/scikit-learn computation (not canned text), build charts, save findings, and export a reproducible Jupyter notebook. No database, no auth, no build step, no framework. Everything server-side lives in one FastAPI process; everything client-side lives in `apps/web/index.html`.

2. The core design bet: never require a full download

This is the idea the entire dataset layer is organized around. Kaggle's download endpoint is just an HTTP GET that streams bytes. Instead of pulling a whole file to disk (what `kagglehub` / the official CLI do), `packages/dataset/src/dataset/streaming.py` does this:

- Lists the dataset's files via `/datasets/list/{owner}/{slug}`.
- Picks the "primary" file by extension priority (csv > tsv > parquet > json > xlsx), then by size.
- **CSV/TSV:** opens a streaming GET, buffers chunks, and stops as soon as it has N bytes (default 5 MB, `ANALYSIS_SAMPLE_MB` env var). Trims the last incomplete line before handing it to pandas.
- **Parquet:** resolves the real signed-URL redirect target, then uses `pyarrow.parquet.ParquetFile` via `fsspec`'s HTTP filesystem, which issues real HTTP range requests to read only as many row groups as needed — and reads the *exact* total row count straight from the file's footer metadata without touching the row data at all.
- **JSON/Excel:** can't be safely truncated (unbalanced brackets / a broken zip container), so these buffer up to 20 MB and fail cleanly above that, telling the user to run a full analysis instead.

Everything downstream — the profiler, the Q&A engine, the charts — operates on this **sample dataframe**, never the full dataset. The `sample` object attached to every analyze response says whether it's a sample and, for byte-truncated formats, extrapolates an estimated total row count as sample_rows × (total_bytes / sample_bytes).

Two escape hatches exist for when the sample genuinely isn't enough:

- `POST /datasets/analyze-full` — downloads to a temp dir via `kagglehub`, profiles the whole thing, deletes the temp dir the moment the request ends.
- `POST /datasets/download-file` — streams the actual Kaggle ZIP straight to the browser.

Neither is on the default path. Both exist purely as opt-in.

3. Request flow — the shape of a typical session

- Browser calls `GET /datasets/search?query=...&limit=150` → `kaggle_search.search_kaggle_datasets` pages through Kaggle's `dataset_list(page=1,2,3...)` (20 results/page) until it hits the limit or runs dry. (Earlier bug, now fixed: this used to pass a count into Kaggle's `max_size` param, which is actually a byte-size *filter* — it silently capped results to ~3–5 datasets.)
- User clicks Analyze → `POST /datasets/analyze {slug}` → the streaming sampler above → `analyze_dataframe()` (the profiler) → response with `dataset` + `analysis`.
- Server-side, the sampled dataframe is stashed in an in-process dict (`apps/api/app/workspace_cache.py`) keyed by the dataset slug — 20-entry LRU cap, 30-minute sliding TTL refreshed on access. This is what lets follow-up questions compute against real data instead of the 300-row JSON `preview` the browser is holding.
- User asks a question → `POST /analysis/question {question, dataset_id}` → server looks up the cached dataframe by dataset_id, falling back to reconstructing a dataframe from the 300-row preview if the cache missed (server restarted, TTL expired) → `qa_engine.answer_question()` → a real computed answer plus the literal pandas/scipy expression that produced it.
- User builds/saves charts and questions as findings, then exports → `POST /analysis/notebook` → `notebook_export.build_notebook()` assembles a `.ipynb` with the sampled data embedded as a literal `pd.DataFrame(records)` cell, live `df.describe()`/`df.isna().sum()` cells, and one markdown+code cell pair per saved finding — either the Q&A engine's literal pandas expression, or generated plotly.express code reconstructing a saved chart's exact type/aggregation/grouping/sort.

4. The Q&A engine — what "AI" actually means here

No LLM is involved. That's a deliberate choice (cost/latency, and keeping the door open to swap in a real LLM backend later without touching the API). It's `27` hand-written intent functions in `packages/analysis/src/analysis/qa_engine.py`, tried in a fixed priority order (most specific first), each of which:

- Checks whether the question contains trigger phrases (`_has_any`) — with a genuinely fiddly correctness detail: single-word triggers must match a *whole token*, never a raw substring, or short words silently match inside unrelated words (`"age"` inside `"percentage"`, `"range"` inside `"orange"`, `"min"` inside `"mining"`). Stemmed triggers (`"correlat*"`, `"predict*"`) use an explicit trailing-* convention rather than a real stemmer.
- If triggered, matches column names mentioned in the question against the dataframe's real columns — `_find_columns` ranks by specificity/name length; `_find_columns_by_position` ranks by order of first mention, used wherever word order carries meaning (e.g. "correlation between X and Y").
- Runs an actual pandas/numpy/scipy/sklearn computation and returns a `QAResult(answer, computed, method, code, value)`.
- If nothing matches, falls through to a fallback that is honestly labeled `computed: false` and suggests example phrasings — it does **not** pretend to answer.

Intent categories, roughly in dispatch order: dataset overview · ML baseline (RandomForest classifier/regressor via sklearn, auto-detects task type from target cardinality, reports accuracy/R^2 against a naive baseline plus top-5 feature importances) · normality (Shapiro–Wilk) · distribution fitting (KS statistic against normal/exponential/uniform/log-normal) · kurtosis · group comparison tests (Welch's t-test or one-way ANOVA depending on group count) · chi-square independence · covariance · ratio · outliers (IQR rule) · percentile/quartile · IQR · range · duplicates · value counts · mode · missing values · uniqueness · skew · correlation · time-based change (two explicit years) · generalized trend (linear regression slope over time) · threshold/percentage/AND-combined filters · column-level ranking ("which column has the highest variance") · top-N/grouped ranking · aggregation (mean/median/sum/etc, optionally grouped) · shape.

> **Correctness traps found during development (worth remembering if you extend this)**
> A binary 0/1 numeric column (like Survived) has to be recognized as a **grouping** variable, not a competing numeric metric — `_categorical_like_cols()` treats any numeric column with ≤20 unique values as categorical-like for this purpose. And scipy 1.18 has a real regression in `kstest`'s string+args dispatch path (throws inside `ndtr`); the fix was passing a distribution object's `.cdf` directly instead of `(name, args)`. Neither of these would show up unless you actually ran real questions against real data — both were found that way, not by inspection.

5. The profiler — deterministic, always-on, not LLM-adjacent

`packages/analysis/src/analysis/profiler.py`'s `analyze_dataframe()` runs on every load regardless of whether the Q&A engine is ever touched. Per column: infers a type (numeric / categorical / datetime / boolean / text / unknown) from dtype plus a cardinality heuristic (≤20 unique values and short average string length ⇒ categorical). Dataset-wide: missingness, duplicate rows, IQR-based outlier count, the top-5 numeric correlation pairs, skew detection (|skew| > 1), naive target-candidate detection, constant/near-constant and high-cardinality (> max(50, 80% of rows)) column flags, and a short list of auto-suggested visualizations and research questions based on which column types are present. All of it becomes the `research_snapshot` block plus `insights` /

`data_quality_issues` lists the frontend renders directly — this layer is useful even with the Q&A box never touched.

6. Frontend — one file, no framework, no build step

`apps/web/index.html` *is* the client: inline `<style>`, inline `<script>`, Plotly loaded from a CDN. State lives in one JS object (`state = {dataset, analysis, slug, findings}`); every UI update is a full re-render of the relevant DOM chunk from that state (`renderAnalysis()`), not component-level reactivity — there is no virtual DOM, no diffing. Charts are built from a `<template>` element (`createChartSlot()`) rather than hardcoded markup, so "+ Add chart" just clones the template again. Each chart slot independently tracks type/x/y/aggregate/group/sort/title and calls `renderChart()` on any change, which either does a client-side groupby+aggregate (a small JS reimplementation of pandas groupby, since this only ever sees the 300-row `preview` the browser already has, not the full server-side sample) or builds a raw per-type Plotly trace directly.

Untrusted third-party text (Kaggle dataset descriptions) is escaped first — routed through `textContent`, not `innerHTML` — before any of the app's own `<p>/<ul>/<strong>` wrapper tags get added back in. That ordering matters: it's specifically there to close off a stored-XSS vector from a malicious dataset description (anyone can publish a Kaggle dataset with an arbitrary description).

7. What's genuinely fragile — not sugarcoated

Single process, in-memory everything

`workspace_cache` is a plain dict behind a `threading.Lock`. Restart the server, lose every cached dataframe (silently falls back to the much-less-accurate 300-row preview). There is no multi-worker/multi-process story at all — two uvicorn workers would each hold a disjoint, inconsistent cache.

No auth, no rate limiting, no multi-user isolation

Anyone who can reach the API can analyze anything, and `workspace_cache` has no per-user namespacing. CORS is wide open (`allow_origins=["*"]`). This is a single-tenant local dev tool, not something to expose publicly as-is.

The Q&A engine is pattern matching, not understanding

It will confidently and correctly answer anything that fits one of the ~27 templates, and honestly decline anything that doesn't. It cannot chain reasoning ("first filter to 2020, then tell me the correlation") — every question is answered from scratch against the whole dataframe, with no memory of prior questions in the conversation.

Sample-based analysis is an approximation by design

A 5 MB sample of a 500 MB file is a statistical sample, not the data — correlations, distributions, and especially rare-category detection can be meaningfully wrong on high-cardinality or long-tailed data. The UI does surface this (the sample banner, the `is_sample` flag), but it's on the user to notice and hit "Run full analysis" when it actually matters.

Client-side chart aggregation only ever sees 300 rows

Regardless of how large the server-side sample actually is, the browser's own preview is capped at 300 rows — so a chart grouping by a category with rare values can visibly disagree with what the Q&A engine (which sees the full cached sample) reports for the same underlying question.

No automated tests

Correctness has been established so far by hand-written ad hoc scripts run against synthetic and live Kaggle data, not a committed test suite. Several real bugs (substring-matching triggers, wrong-column selection in stats tests, the scipy kstest regression) were only caught this way — there's nothing stopping a regression on the next change from going unnoticed.

Heavy dependency footprint for what it does

`pyarrow` + `fsspec` + `aiohttp` (for parquet range reads) and `scipy` + `scikit-learn` (for the Q&A engine's stats/ML) are all real, non-trivial dependencies pulled in to support what is still fundamentally a single-file frontend and a thin FastAPI backend.

8. File map — where to actually go looking

Path	What it is
`apps/api/app/main.py`	Every FastAPI route. Thin orchestration only — no business logic lives here.
`apps/api/app/workspace_cache.py`	The in-memory TTL dataframe cache described in §3.
`apps/web/index.html`	The entire frontend: markup, CSS, and JS in one file.
`packages/dataset/.../streaming.py`	The no-full-download sampling logic for CSV/parquet/json/xlsx (§2).
`packages/dataset/.../kaggle_search.py`	Kaggle auth, paginated search, full-download fallback, dataset description fetch.
`packages/dataset/.../loader.py`	Local-path and in-memory (upload) file parsing.
`packages/dataset/.../descriptions.py`	Heuristic column-description extraction from a dataset's free-text description.
`packages/analysis/.../profiler.py`	analyze_dataframe() — the research snapshot (§5).
`packages/analysis/.../qa_engine.py`	The 27-intent deterministic NLU engine (§4).
`packages/analysis/.../notebook_export.py`	Builds the exported .ipynb from the sample + saved findings.

9. Endpoint reference

Endpoint	Purpose
`GET /health`	Liveness check.
`GET /auth/kaggle`	Whether Kaggle credentials are configured and how.
`GET /datasets/search`	Paginated Kaggle dataset search (up to ~300 results).
`POST /datasets/analyze`	Default path: sample-based analysis, never a full download.
`POST /datasets/analyze-full`	Opt-in: full download to a temp dir, full-precision analysis, then delete.
`POST /datasets/download-file`	Opt-in: streams the raw Kaggle ZIP to the browser.
`POST /datasets/upload`	Analyzes a local file the user picks; parsed in memory, never written to disk.
`POST /datasets/load`	Loads from a local server-side path; dev/debug use.
`GET /datasets/demo`	Small hardcoded demo dataset, no Kaggle call.
`POST /analysis/question`	The Q&A engine entry point (§4).
`POST /analysis/notebook`	Builds and returns the exported .ipynb JSON.

10. If you extend this later

- **Swapping the Q&A engine for a real LLM:** keep the `(question, dataframe, snapshot) -> QAResult` contract in `qa_engine.answer_question` intact. `main.py`'s `/analysis/question` route doesn't need to change — just which engine it calls, gated by an env var (`AI_PROVIDER=deterministic|openai|anthropic` was the placeholder discussed).
- **Surviving a restart / scaling past one process:** `workspace_cache` needs to move to something external — Redis, or even a temp SQLite file. Right now it's deliberately the simplest thing that works for a single local dev session, nothing more.
- **Real multi-user support:** the dataset_id namespacing in workspace_cache would need a user/session prefix, and CORS (`allow_origins=["*"]`) would need tightening from wide-open to an actual allowlist.

Generated as a working reference, not a spec — if the code and this document ever disagree, trust the code.